

Travel Planner Agent

A Multi-Step LLM Pipeline for Intelligent Trip Planning

Repository: github.com/k-1208/travel_agent_pipeline

1. Problem Statement

Planning a trip involves a chain of interdependent decisions that resist compression into a single prompt. A request such as *"3-day trip to Goa under Rs. 20,000"* requires gathering real-world pricing data, reasoning across accommodation, transport and activities, producing a coherent schedule, and validating that schedule against hard budget constraints. A single LLM call applied to all of this simultaneously produces output that is either too vague to act on or subtly inconsistent, for example a polished itinerary whose costs silently exceed the stated budget.

The Travel Planner Agent decomposes this problem into six sequential steps where each step receives structured output from the prior one. Multi-step chaining adds genuine value for three reasons: budget allocation requires real prices that only a tool call can supply; critique is cognitively distinct from generation, since the same model that drafted the itinerary tends to rationalise its own output; and a traceable chain lets an evaluator pause at any node, inspect its input and output, and verify that the reasoning held.

2. Chain Design

Step 1 – LLM (Decompose) `src/agents/step1_decompose.py`

Receives the raw user query and produces a typed JSON object: destination, duration, total budget in INR, traveller count, preferences and inferred dates. This step exists because every downstream step requires clean, typed fields; a budget written as "Rs. 20k" must become the integer 20000 before it can participate in arithmetic comparisons in Steps 3 and 6.

Step 2 – Tool (Fetch) `src/tools/hotels.py | flights.py | activities.py`

Calls three external APIs in sequence: Amadeus Hotel Search, Amadeus Flight Offers, and OpenTripMap for activities. All three implement a graceful fallback to curated mock data when API keys are absent, so the chain never breaks during development. Outputs are structured lists stored in the shared `AgentState` object and passed to every subsequent LLM prompt.

Step 3 – LLM (Budget Allocation) `src/agents/step2_budget.py`

Receives trip parameters from Step 1 alongside real prices from Step 2 and produces an approved budget split across flights, hotels, activities, food, local transport and a contingency buffer. Without Step 2, the model would estimate costs from training data that may be months out of date.

Step 4 – LLM (Itinerary Generation) `src/agents/step3_itinerary.py`

Takes all prior outputs and produces a day-by-day schedule with time slots, activity names, location notes and per-slot cost estimates. The approved budget from Step 3 must exist before activities can be selected; otherwise the schedule might appear reasonable while silently overspending in one or more categories.

Step 5 – LLM (Critique) `src/agents/step4_critique.py`

Evaluates the draft itinerary against the original constraints using an adversarial critic persona, flagging issues by severity (*high*, *medium*, *low*). Separating critique from generation prevents the model from rationalising its own draft output.

Step 6 – LLM (Optimise) `src/agents/step5_optimize.py`

Applies every high-severity fix and as many medium-severity fixes as the budget permits, logging each change in a `changes_made` field. Keeping this as a distinct step means critique and repair can each be inspected and explained independently during the live demo.

3. Tool Integration

Step 2 is the sole tool-call step, making three sequential API requests. The Amadeus Hotel Search and Flight Offers APIs were chosen because they are available on a free development tier, return a consistent JSON schema, and cover Indian domestic routes natively. OpenTripMap was selected for activities because its category tags map directly onto user preferences such as *beach*, *culture* and *adventure*.

Tool outputs enter the chain as plain Python lists of dictionaries stored in `AgentState`. Step 3's prompt surfaces actual hotel names with their per-night price and flight options with their per-person cost verbatim in the user message, so the LLM cannot substitute hallucinated figures. If the cheapest Goa hotel from Amadeus costs Rs. 2,800 per night, that number appears literally in Step 3's input and the resulting budget plan must account for it. The same real data flows into Step 4's scheduling and Step 5's critique, maintaining factual consistency across the full chain.

4. Limitations

- **IATA code mapping.** The project uses a hand-coded dictionary of fifteen Indian cities. Any destination outside this set, including Leh, Andaman or Tirupati, falls back to generic mock data without warning. The correct fix is a live lookup against the Amadeus Locations API, which resolves a city name to its IATA code in one call, but adding it would have required restructuring Step 2 to run a pre-query before the main searches.
- **Absent SerpAPI summarisation.** The original design included a sub-step fetching and condensing recent travel content to supply fresher destination context into Step 4's prompt. Without it, qualitative recommendations rely on the model's training data, which may be months out of date for rapidly changing destinations.
- **JSON parsing fragility.** Deeply nested output from Step 6 occasionally causes parsing failures that are not surfaced clearly to the user, despite the three-attempt retry loop with exponential back-off.

5. Reflection and Future Enhancements

The most consequential design decision was separating critique and optimisation. During testing, a combined "review and revise" call consistently acknowledged issues in prose but left the schedule unchanged. A dedicated repair call with a narrow, fix-only instruction produced measurably more consistent results, and this pattern would not have emerged without building the chain from scratch rather than relying on a framework abstraction.

Given more time, the highest priority would be the **SerpAPI summarisation step**, injecting destination context fresher than training data can provide. The second priority is a **live IATA resolver** via the Amadeus Locations API, removing the fifteen-city ceiling. A **user confirmation step** after budget allocation would allow travellers to adjust the split before the itinerary is generated, moving the agent toward an interactive model. Finally, adopting a **structured-output mode** available in newer Grok model versions would replace the fragile JSON retry logic entirely and make the chain production-ready.

References omitted per assignment instructions.